

Robotics Integration Group Project 1

Lab 1 Experiment Report: Linux, C++ and Git Setup, plus RandomVector

Huang Boyu 24020036026.

Ocean University of China, 24th Cohort Sino-Foreign CS Class 1.

Lab date: 27th September 2026. Report date: 27th September 2026.

Summary

This report covers the whole Lab 1 workflow of the Robotics Integration Group Project: creating and logging into an Ubuntu 20.04.5 ARM64 virtual machine, backing up and switching the apt sources to the Tsinghua mirror, installing the essential toolchain (build-essential / CMake / Git), practicing basic Linux commands, Git version management, single-file C++ compilation with g++, a multi-file CMake project (`lab1_demo`), cloning the course repository (`ouc-vlab-course/vnav-codes`) and the personal submission repository (`hbyhby123/vnav-personal`), the Shell exercises (counting lines/words/non-blank lines of `dante.txt`, appending five `fortune` outputs), the C++ warm-up questions, and finally implementing all six member functions of the `RandomVector` class. The final program runs with the fixed seed `std::srand(314159)` and prints 20 random values, Mean 0.393617, Min 0.0667949, Max 0.999018 and a vertical `***` histogram with five min-max bins whose counts are 8, 1, 8, 2, 1 — exactly matching the example output on the course website, which confirms the implementation is correct and reproducible.

Objectives and Environment

The goal of Lab 1 is to build the basic development workflow used throughout the course: operate files in Linux, manage code with Git, compile with g++ or CMake, and read, modify and verify C++ programs from the course repositories.

Table 1: Actual experiment environment.

Item	Actual environment
Hardware	MacBook Air (Apple silicon), VMware Fusion virtual machine
Operating system	Ubuntu 20.04.5 LTS, aarch64 (ARM64), hostname <code>ubuntu2004</code>
C/C++ compiler	gcc 9.4.0 / g++ 9.4.0
Build tool	CMake 3.16.3
Version control	Git 2.25.1
Course repository	https://github.com/ouc-vlab-course/vnav-codes
Personal repository	https://github.com/hbyhby123/vnav-personal
Code baseline	MIT-SPARK/VNAV-labs (<code>lab1</code> folder)

Figure 01: Ubuntu 20.04.5 LTS / aarch64 confirmed with `lsb_release -d` and `uname -m`; gcc, g++, cmake and git versions.

Part One: Ubuntu 20.04 VM Installation

A new virtual machine was created in VMware Fusion from the `ubuntu-20.04.5-live-server-arm64.iso` image. After the first boot the system shows a terminal login prompt; after logging in and finishing the required configuration, the graphical desktop works normally. The course suggests NAT networking with host connection and DHCP enabled; in this environment GitHub and the apt mirrors were reachable, which proves the network works, although the individual settings were not shown separately.

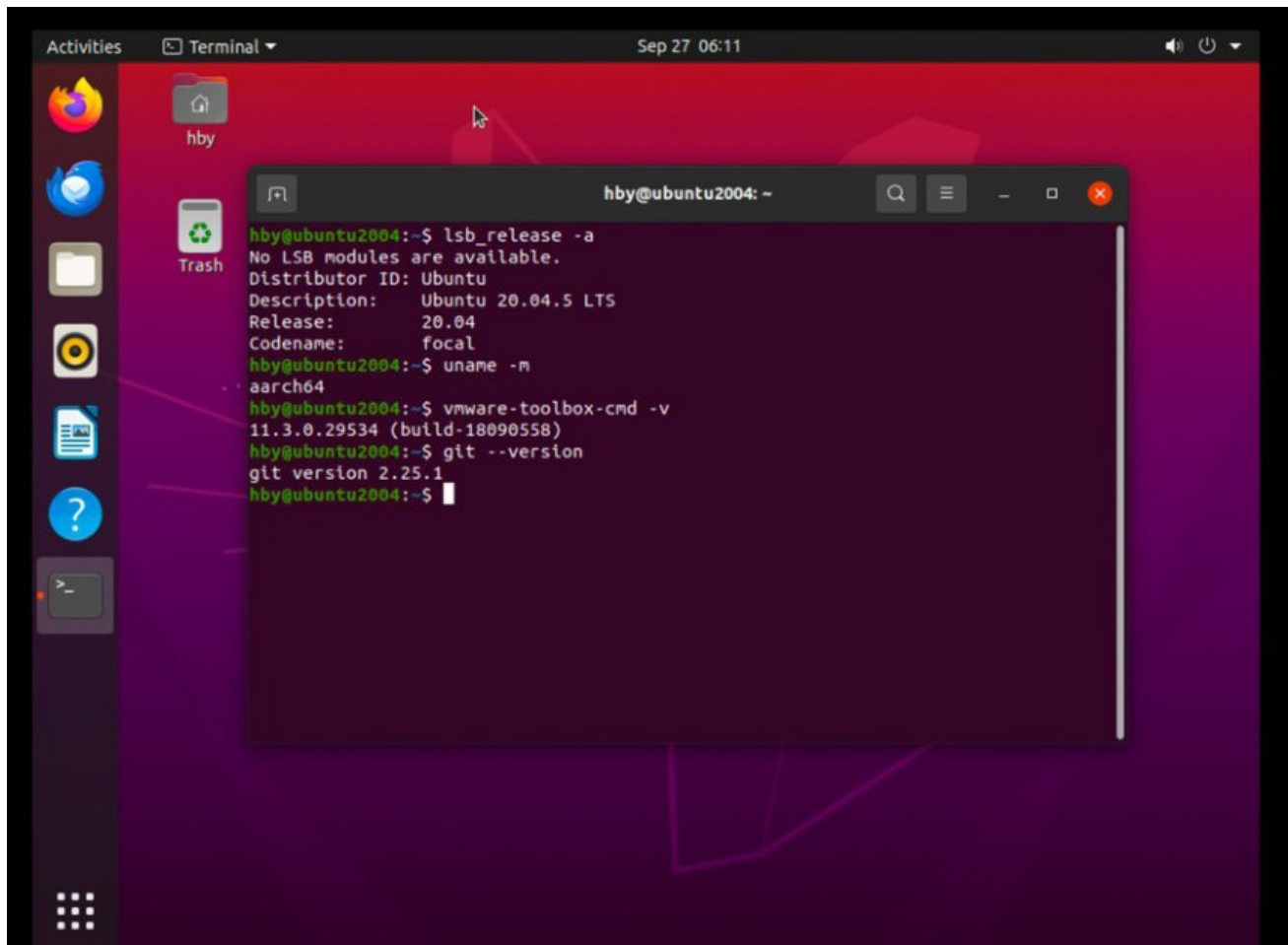
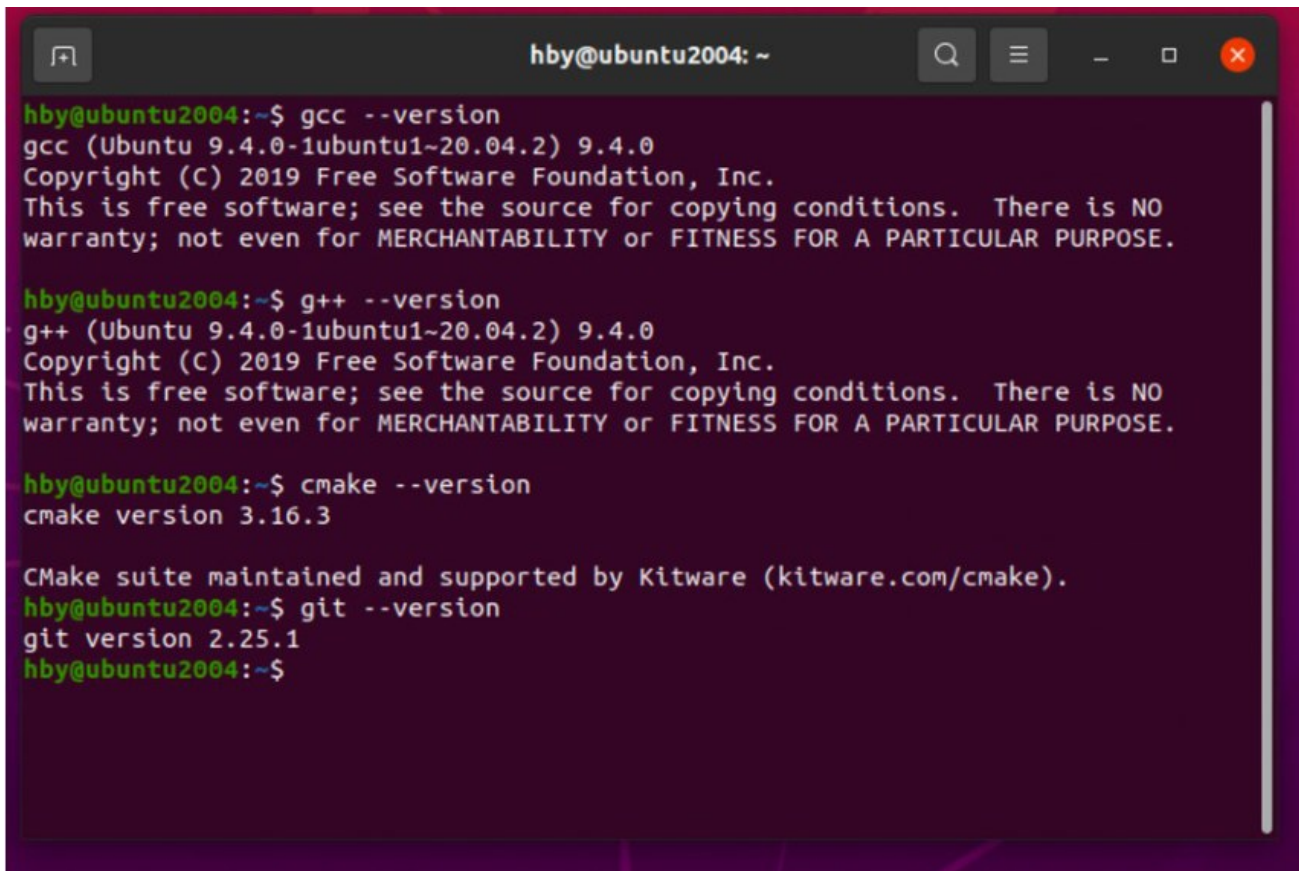


Figure 1: Ubuntu version, architecture and toolchain checks



```
hby@ubuntu2004: ~  
hby@ubuntu2004:~$ gcc --version  
gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0  
Copyright (C) 2019 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
hby@ubuntu2004:~$ g++ --version  
g++ (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0  
Copyright (C) 2019 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
hby@ubuntu2004:~$ cmake --version  
cmake version 3.16.3  
  
CMake suite maintained and supported by Kitware (kitware.com/cmake).  
hby@ubuntu2004:~$ git --version  
git version 2.25.1  
hby@ubuntu2004:~$
```

Figure 2: Toolchain versions

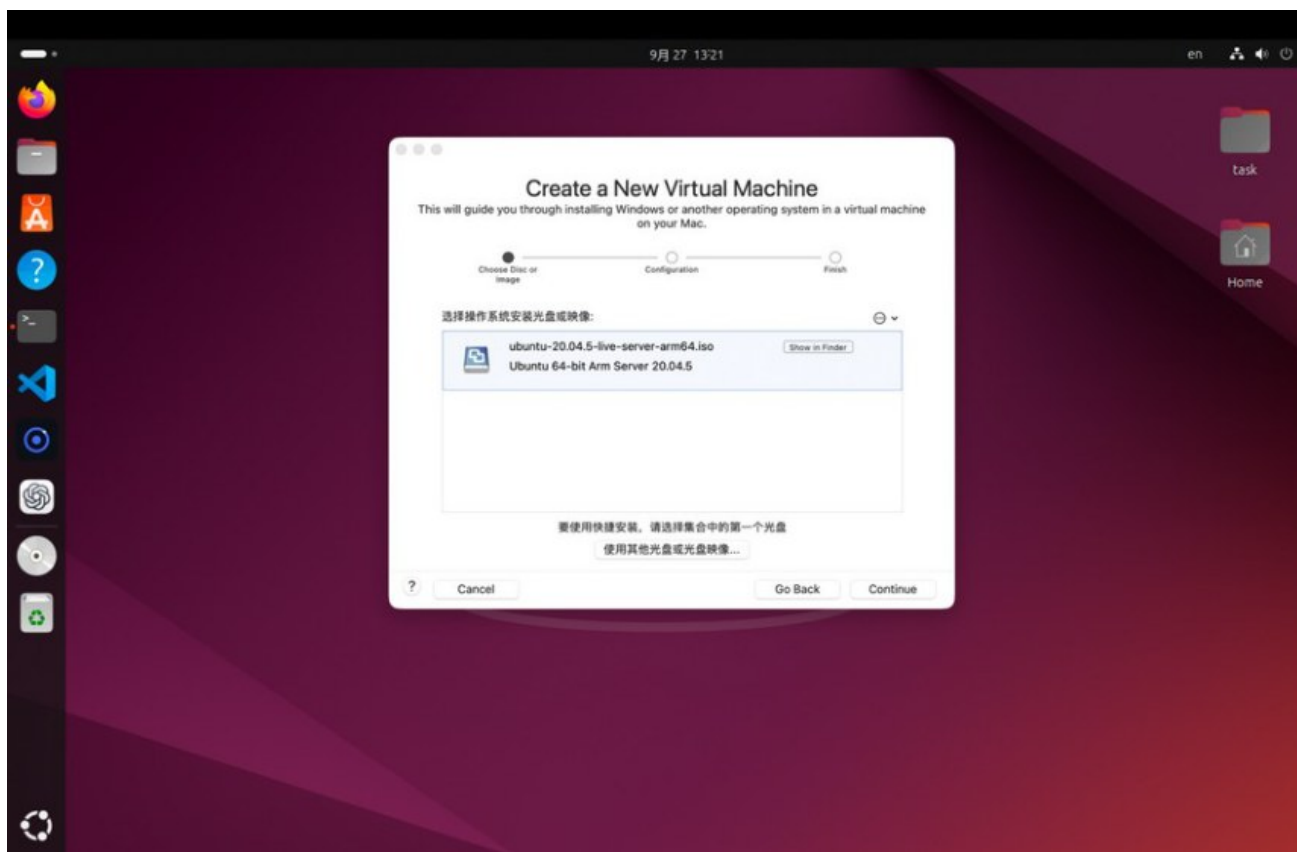


Figure 3: Creating the VM

```

Eb16pmvTFXbWnS3Dh99RV3x2NbiuFMj5eEoy8m43mSDoqaHiMX91HSnukXDgBAJti98yhuj4/u6DPiPcFTaHV7sjIvg1DMRGRnungdYN607Yvqp11ihp741FsEw1tj7n
MdoKm5uPfZT2Jrrz8FHfFkHly9zMOHEc615ic1YUmyYK1H2WmqPAH8XNg3Md/b6i11HQHfQ1V818LuLQSNmfIv4FJdIxmKcGMNpkGAsZm8FKVKMzX009SBIfz1QSMkhM
3Nje1XF1Iz3NMvKn06shUnU0UnU/E/+ /SnZf81U= root@ubuntu2004
-----END SSH HOST KEY KEYS-----
[ 15.591121] cloud-init[1882]: Cloud-init v. 23.1.2-0ubuntu0~20.04.2 running 'modules:final' at Sun, 27 Sep 2026 05:50:14 +000. Up 15.54 seconds.
[ 15.591188] cloud-init[1882]: Cloud-init v. 23.1.2-0ubuntu0~20.04.2 finished at Sun, 27 Sep 2026 05:50:14 +0000. Datasource DataSourceNone. Up 15.58 seconds
[ 15.591239] cloud-init[1882]: 2026-09-27 05:50:14,760 - cc_final_message.py[WARNING]: Used fallback datasource
[ OK ] Finished Execute cloud user/final scripts.
[ OK ] Reached target Cloud-init target.

ubuntu2004 login: 123
Password:

Login incorrect
ubuntu2004 login: hby
Password:
Welcome to Ubuntu 20.04.5 LTS (GNU/Linux 5.4.0-216-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

System information as of Sun 27 Sep 2026 05:51:17 AM UTC

System load:  0.13           Processes:      241
Usage of /:   34.4% of 18.01GB Users logged in: 0
Memory usage: 2%            IPv4 address for enp2s0: 192.168.2.131
Swap usage:   0%

71 updates can be applied immediately.
3 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hby@ubuntu2004:~$

```

Figure 4: First terminal login

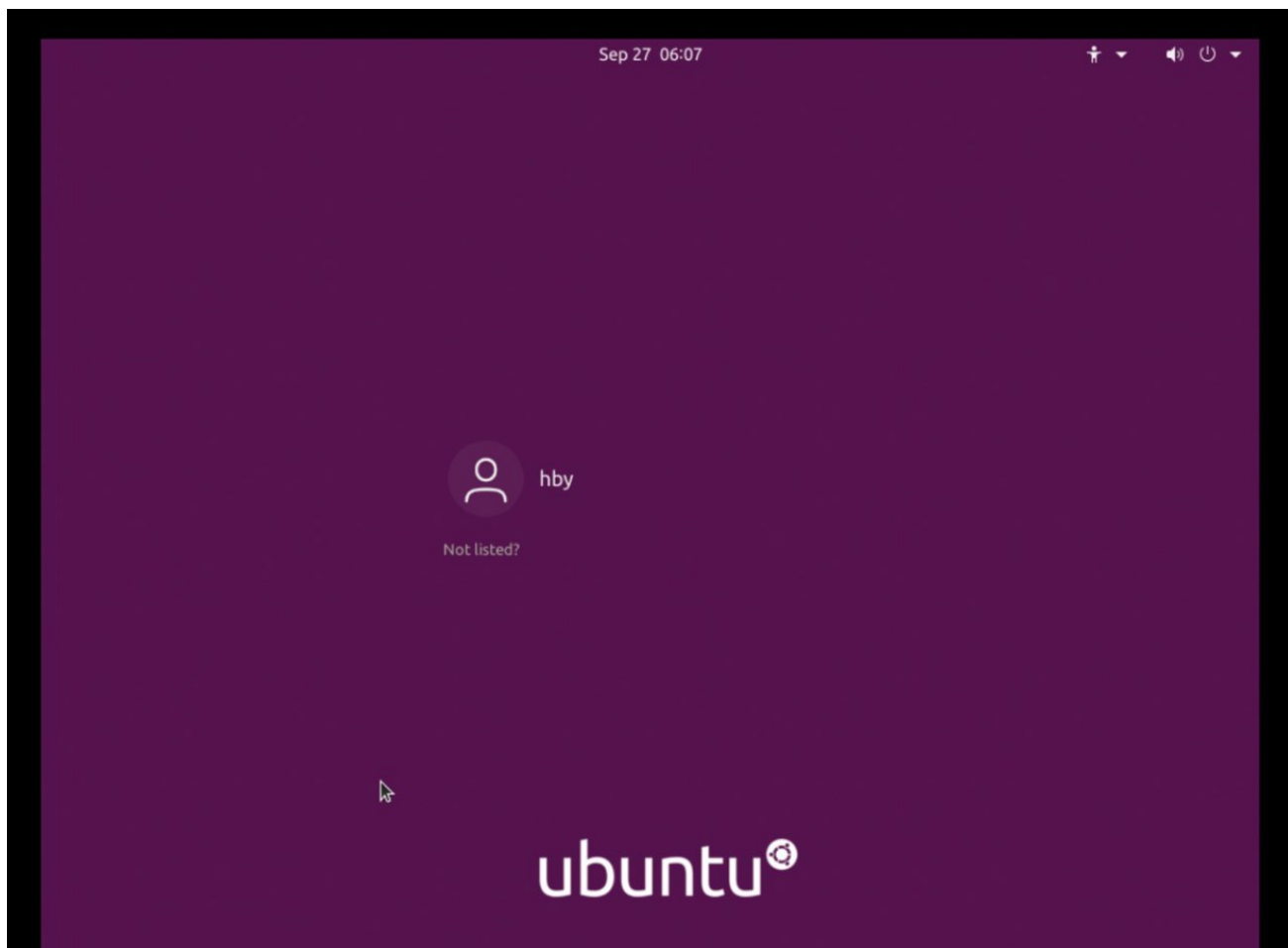


Figure 5: Graphical desktop login

Figure 02: Creating the VM from the ARM64 image; first login at the text console; the graphical login screen after reboot.

Part One: Backup and Switch to the Tsinghua Mirror

The effective Ubuntu sources were originally in `/etc/apt/sources.list.d/original.list` using the official `ubuntu-ports` addresses. Both files were backed up first, then the mirror addresses were rewritten into `sources.list` with `sed` and the old configuration was disabled to avoid duplicate entries:

```
sudo cp -n /etc/apt/sources.list /etc/apt/sources.list.bak
sudo cp -n /etc/apt/sources.list.d/original.list \
/etc/apt/sources.list.d/original.list.bak
sudo sed 's|http://ports.ubuntu.com/ubuntu-ports/|https://mirrors.tuna.tsinghua.edu.cn/ubuntu-ports/|g' \
/etc/apt/sources.list.d/original.list | sudo tee /etc/apt/sources.list
sudo mv /etc/apt/sources.list.d/original.list \
/etc/apt/sources.list.d/original.list.disabled
sudo apt update
```

The release stays focal, with the `focal-updates`, `focal-backports` and `focal-security` components. Excerpt of the update output:

```
Get:1 https://mirrors.tuna.tsinghua.edu.cn/ubuntu-ports focal InRelease
Get:2 https://mirrors.tuna.tsinghua.edu.cn/ubuntu-ports focal-updates InRelease
Get:3 https://mirrors.tuna.tsinghua.edu.cn/ubuntu-ports focal-backports InRelease
Get:4 https://mirrors.tuna.tsinghua.edu.cn/ubuntu-ports focal-security InRelease
Fetched 45.2 MB in 12s (3,826 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
60 packages can be upgraded.
```

The package index was refreshed successfully. “60 packages can be upgraded” is only a status hint, not a failure; no system upgrade was performed in this step. `sources.list.bak` is the backup of the old `sources.list`, while the real old Ubuntu sources are kept as `original.list.bak`.

Part Two: Essential Tools

```
sudo apt install build-essential cmake git
```

`dpkg-query` confirms all three packages are `install ok installed`:

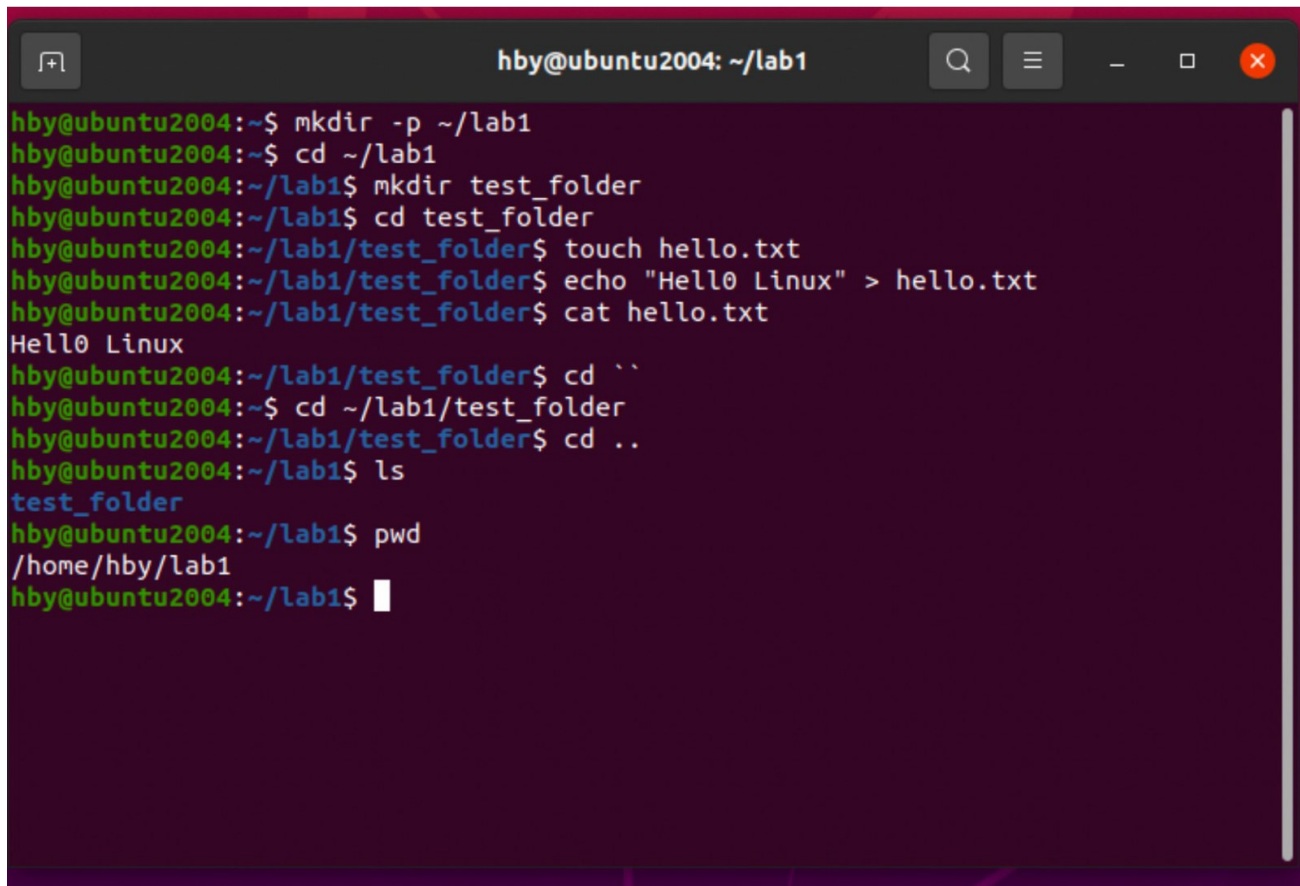
Package	Version
build-essential	12.8ubuntu1.1
cmake	3.16.3-1ubuntu1.20.04.1
git	1:2.25.1-1ubuntu3.14

`build-essential` provides the basic compiler toolchain, CMake generates the build files of a project, and Git manages versions. The g++ compilation of `RandomVector` and the CMake build of `lab1_demo` below additionally prove that the toolchain works.

Part Two: Linux Basic Commands

In `~/lab1` the basic filesystem operations were practiced: creating folders, entering directories, creating files, writing text and reading it back.

Command	Meaning	Use in this lab
ls	List directory contents	Check that files/folders were created
cd	Change directory	Enter <code>lab1</code> , <code>test_folder</code> , etc.
pwd	Print working directory	Confirm the current path
mkdir	Make directory	Create the working directories
touch	Create an empty file	Create <code>hello.txt</code>
echo	Print / write text	Write <code>Hello Linux</code> into <code>hello.txt</code>
cat	Show file contents	Check what was written



```

hby@ubuntu2004: ~/lab1
hby@ubuntu2004:~$ mkdir -p ~/lab1
hby@ubuntu2004:~$ cd ~/lab1
hby@ubuntu2004:~/lab1$ mkdir test_folder
hby@ubuntu2004:~/lab1$ cd test_folder
hby@ubuntu2004:~/lab1/test_folder$ touch hello.txt
hby@ubuntu2004:~/lab1/test_folder$ echo "Hello Linux" > hello.txt
hby@ubuntu2004:~/lab1/test_folder$ cat hello.txt
Hello Linux
hby@ubuntu2004:~/lab1/test_folder$ cd ..
hby@ubuntu2004:~/lab1$ ls
test_folder
hby@ubuntu2004:~/lab1$ pwd
/home/hby/lab1
hby@ubuntu2004:~/lab1$

```

Figure 6: Linux file and directory operations

Figure 03: `mkdir test_folder`, `touch hello.txt`, `echo "Hello Linux" > hello.txt`, `cat hello.txt` and `ls` all work as expected.

Part Three: Git Basics

In the `lab1_ws` directory a local repository was initialized with `git init`, a `README.md` test file was created, and the first commit required the global `user.name` / `user.email` configuration. Afterwards the commit succeeded and `git log --oneline` shows the history.

```

git init
git config --global user.name "hby"
git config --global user.email "..." # fill in the email as prompted

```

```
git add README.md
git commit -m "first commit"
git log --oneline
```

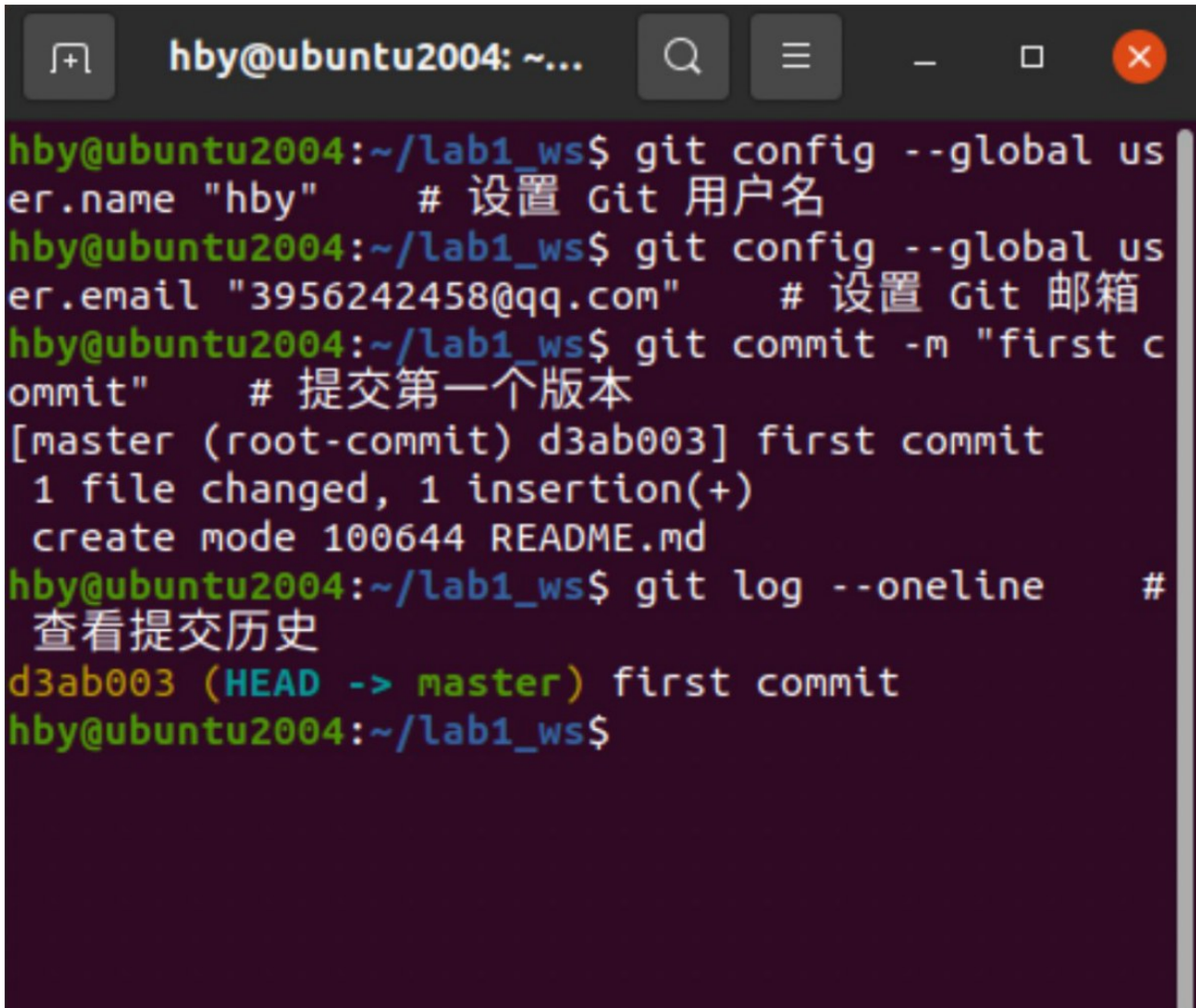
A terminal window titled 'hby@ubuntu2004: ~...' with standard window controls. The terminal shows the following commands and output:
hby@ubuntu2004:~/lab1_ws\$ git config --global user.name "hby" # 设置 Git 用户名
hby@ubuntu2004:~/lab1_ws\$ git config --global user.email "3956242458@qq.com" # 设置 Git 邮箱
hby@ubuntu2004:~/lab1_ws\$ git commit -m "first commit" # 提交第一个版本
[master (root-commit) d3ab003] first commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
hby@ubuntu2004:~/lab1_ws\$ git log --oneline # 查看提交历史
d3ab003 (HEAD -> master) first commit
hby@ubuntu2004:~/lab1_ws\$

Figure 7: Git configuration, first commit and history

Figure 04: Git user configuration, the first commit and its history.

The full workflow is: clone/init to get a repository → status/diff to inspect changes → add to stage → commit to save a version → push to sync with the remote. Git is not only a “download tool”: it records the modification history so that changes can be reviewed and rolled back, which is essential for teamwork.

Part Three: C++ Single-File Compilation

The minimal compile-run loop was tested with `hello.cpp`:

```
// hello.cpp
#include <iostream>
int main() {
    std::cout << "Hello C++" << std::endl;
```

```

    return 0;
}

g++ hello.cpp -o hello
./hello
# Hello C++

```

Figure 05: g++ compiles `hello.cpp` into `hello` and the program prints `Hello C++`.

Part Three: CMake Multi-File Project

First a trivial `cmake_test` project was configured, built and run (`cmake . && make && ./hello`). Then the multi-file project `~/vnav-personal/lab1/lab1_demo` was created following the teacher's example, with `CMakeLists.txt`, `main.cpp`, `my_lib.hpp` and `my_lib.cpp`:

```

cmake_minimum_required(VERSION 3.10)
project(hello_world VERSION 1.0 LANGUAGES CXX)
add_library(mylib my_lib.cpp my_lib.hpp)
add_executable(main main.cpp)
target_link_libraries(main PRIVATE mylib)

// main.cpp
#include <iostream>
#include "my_lib.hpp"
int main() {
    std::cout << "Hello world!" << std::endl;
    std::cout << my_lib_function() << std::endl;
    return 0;
}

mkdir -p build
cd build
cmake .. && make && ./main
# Hello world!
# In library

```

Figure 06: Writing `CMakeLists.txt` for the simple test project; the multi-file project builds and prints `Hello world!` and `In library`.

For single files, calling `g++` directly is the quickest option; for real projects CMake manages sources, targets and the linking relationship centrally, which is why robot projects use it.

Part Four: Course and Personal Repositories

The MIT-SPARK/VNAV-labs repository (labs 1–9) was first cloned with the standard HTTPS address (an earlier malformed address asked for a username/password; the correct one fixed it). Then the course repository was cloned as required:

```

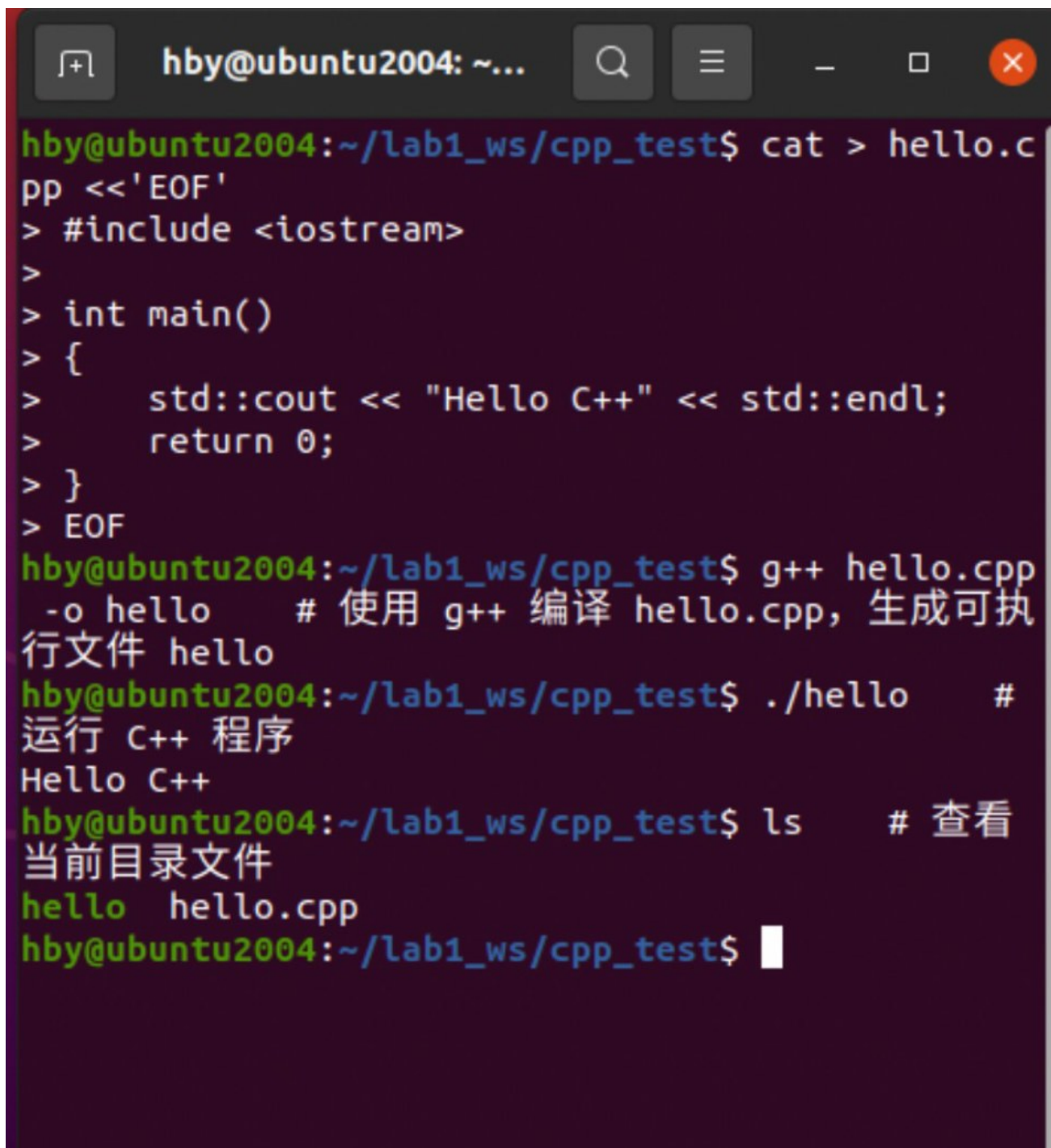
git clone https://github.com/MIT-SPARK/VNAV-labs.git
git clone https://github.com/ouc-vlab-course/vnav-codes.git ~/vnav-codes

```

Figure 07: MIT VNAV-labs cloned successfully; the course repository clone receives 593 objects and resolves 143 deltas.

The course code lives in `/home/hby/vnav-codes`; its `lab1` folder contains `main.cpp`, `random_vector.cpp` and `random_vector.h`.

Figure 08: Listing the course repository; reading `main.cpp` (the fixed seed `std::srand(314159)`) and the `RandomVector` interface.

A terminal window titled 'hby@ubuntu2004: ~...' with standard window controls. The terminal shows the creation of a C++ file 'hello.cpp' using 'cat', its compilation with 'g++' into an executable 'hello', and its execution with './hello', which outputs 'Hello C++'. Finally, 'ls' is used to list the files in the directory, showing 'hello' and 'hello.cpp'.

```
hby@ubuntu2004:~/lab1_ws/cpp_test$ cat > hello.cpp <<'EOF'
> #include <iostream>
>
> int main()
> {
>     std::cout << "Hello C++" << std::endl;
>     return 0;
> }
> EOF
hby@ubuntu2004:~/lab1_ws/cpp_test$ g++ hello.cpp
-o hello      # 使用 g++ 编译 hello.cpp, 生成可执行文件 hello
hby@ubuntu2004:~/lab1_ws/cpp_test$ ./hello      # 运行 C++ 程序
Hello C++
hby@ubuntu2004:~/lab1_ws/cpp_test$ ls      # 查看当前目录文件
hello  hello.cpp
hby@ubuntu2004:~/lab1_ws/cpp_test$
```

Figure 8: Single-file C++ compilation and run

```
hby@ubuntu2004: ~/lab1_ws/cmake_test
>     return 0;
> }
> EOF
hby@ubuntu2004:~/lab1_ws/cmake_test$ cat > CMakeLists.txt <<'EOF'
> cmake_minimum_required(VERSION 3.10)
>
> project(cmake_test)
>
> add_executable(hello main.cpp)
> EOF
hby@ubuntu2004:~/lab1_ws/cmake_test$ ls
CMakeLists.txt  main.cpp
hby@ubuntu2004:~/lab1_ws/cmake_test$ cmake .
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/hby/lab1_ws/cmake_test
hby@ubuntu2004:~/lab1_ws/cmake_test$ make
Scanning dependencies of target hello
[ 50%] Building CXX object CMakeFiles/hello.dir/main.cpp.o
[100%] Linking CXX executable hello
[100%] Built target hello
hby@ubuntu2004:~/lab1_ws/cmake_test$ ./hello
Hello CMake
hby@ubuntu2004:~/lab1_ws/cmake_test$
```

Figure 9: Simple cmake_test configuration

```
hby@ubuntu2004: ~/vnav-personal/lab1/lab1_demo/build
> EOF
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$ cat > my_lib.hpp <<'EOF'
> #pragma once
> #include <string>
>
> std::string my_lib_function();
> EOF
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$ cat > my_lib.cpp <<'EOF'
> #include <string>
> #include "my_lib.hpp"
>
> std::string my_lib_function() {
>     return "In library";
> }
> EOF
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$ ls
build CMakeLists.txt main.cpp my_lib.cpp my_lib.hpp
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$ cd ~/vnav-personal/lab1/lab1_demo
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$ mkdir -p build
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo$ cd build
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo/build$ cmake .. && make && ./main
-- Configuring done
-- Generating done
-- Build files have been written to: /home/hby/vnav-personal/lab1/lab1_demo/build
Scanning dependencies of target mylib
[ 25%] Building CXX object CMakeFiles/mylib.dir/my_lib.cpp.o
[ 50%] Linking CXX static library libmylib.a
[ 50%] Built target mylib
Scanning dependencies of target main
[ 75%] Building CXX object CMakeFiles/main.dir/main.cpp.o
[100%] Linking CXX executable main
[100%] Built target main
Hello world!
In library
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo/build$
```

Figure 10: Multi-file build success

```
hby@ubuntu2004: ~/lab1_ws
hby@ubuntu2004:~/lab1_ws$ git clone https://github.com/MIT-SPARK/VNAV-labs.git # 下载 MIT VNAV 实验代码
Cloning into 'VNAV-labs'...
remote: Enumerating objects: 916, done.
remote: Counting objects: 100% (121/121), done.
remote: Compressing objects: 100% (64/64), done.
Receiving objects: 43% (394/916), 476.00 KiB |Receiving objects: 44% (404/916), 476.00 KiB |Receiving objects: 45% (413/916), 476.00 KiB |Receiving objects: 46% (422/916), 476.00 KiB |Receiving objects: 47% (431/916), 476.00 KiB |Receiving objects: 48% (440/916), 476.00 KiB |Receiving objects: 49% (449/916), 476.00 KiB |Receiving objects: 50% (458/916), 476.00 KiB |Receiving objects: 51% (468/916), 476.00 KiB |Receiving objects: 52% (477/916), 476.00 KiB |Receiving objects: 53% (486/916), 476.00 KiB |Receiving objects: 54% (495/916), 476.00 KiB |Receiving objects: 55% (504/916), 476.00 KiB |Receiving objects: 56% (513/916), 476.00 KiB |Receiving objects: 57% (523/916), 476.00 KiB |Receiving objects: 58% (532/916), 476.00 KiB |Receiving objects: 59% (541/916), 476.00 KiB |Receiving objects: 60% (550/916), 476.00 KiB |Receiving objects: 61% (559/916), 476.00 KiB |Receiving objects: 62% (568/916), 476.00 KiB |Receiving objects: 63% (578/916), 476.00 KiB |Receiving objects: 64% (587/916), 476.00 KiB |Receiving objects: 65% (596/916), 476.00 KiB |Receiving objects: 66% (605/916), 476.00 KiB |Receiving objects: 67% (614/916), 476.00 KiB |Receiving objects: 68% (623/916), 476.00 KiB |Receiving objects: 69% (633/916), 476.00 KiB |Receiving objects: 70% (642/916), 476.00 KiB |Receiving objects: 71% (651/916), 476.00 KiB |Receiving objects: 72% (660/916), 476.00 KiB |Receiving objects: 73% (669/916), 476.00 KiB |Receiving objects: 74% (678/916), 476.00 KiB |Receiving objects: 75% (687/916), 476.00 KiB |Receiving objects: 76% (697/916), 476.00 KiB |Receiving objects: 77% (706/916), 476.00 KiB |Receiving objects: 78% (715/916), 476.00 KiB |Receiving objects: 79% (724/916), 476.00 KiB |Receiving objects: 80% (733/916), 476.00 KiB |Receiving objects: 81% (742/916), 476.00 KiB |remote: Total 916 (delta 71), reused 57 (delta 57), pack-reused 795 (from 1)
Receiving objects: 82% (752/916), 476.00 KiB |Receiving objects: 83% (761/916), 476.00 KiB |Receiving objects: 84% (770/916), 476.00 KiB |Receiving objects: 85% (779/916), 476.00 KiB |Receiving objects: 86% (788/916), 476.00 KiB |Receiving objects: 87% (797/916), 476.00 KiB |Receiving objects: 88% (807/916), 476.00 KiB |Receiving objects: 89% (816/916), 476.00 KiB |Receiving objects: 90% (825/916), 476.00 KiB |Receiving objects: 91% (834/916), 476.00 KiB |Receiving objects: 92% (843/916), 476.00 KiB |Receiving objects: 93% (852/916), 476.00 KiB |Receiving objects: 94% (862/916), 476.00 KiB |Receiving objects: 95% (871/916), 476.00 KiB |Receiving objects: 96% (880/916), 476.00 KiB |Receiving objects: 97% (889/916), 476.00 KiB |Receiving objects: 98% (898/916), 476.00 KiB |Receiving objects: 99% (907/916), 476.00 KiB |Receiving objects: 100% (916/916), 476.00 KiB |Receiving objects: 100% (916/916), 2.42 MiB | 2.70 MiB/s, done.
Resolving deltas: 100% (289/289), done.
hby@ubuntu2004:~/lab1_ws$ ls
snake_test  cpp_test  README.md  VNAV-labs
hby@ubuntu2004:~/lab1_ws$
```

Figure 11: Cloning MIT VNAV-labs

```
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo/build$ git clone https://github.com/ouc-vlab-course/vnav-codes.git ~/vnav-codes
Cloning into '/home/hby/vnav-codes'...
remote: Enumerating objects: 593, done.
remote: Counting objects: 100% (55/55), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 593 (delta 31), reused 26 (delta 26), pack-reused 538 (from 1)
Receiving objects: 100% (593/593), 2.33 MiB | 2.11 MiB/s, done.
Resolving deltas: 100% (143/143), done.
hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo/build$
```

Figure 12: Cloning the course repository

```
hby@ubuntu2004:~/lab1_ws/VNAV-labs$ ls
lab1  lab3  lab5  lab7  lab9
lab2  lab4  lab6  lab8  README.md
hby@ubuntu2004:~/lab1_ws/VNAV-labs$
```

Figure 13: Course repo directory listing

```
hby@ubuntu2004: ~/lab1_ws/VNAV-labs/lab1
hby@ubuntu2004:~/lab1_ws/VNAV-labs$ ls
lab1 lab3 lab5 lab7 lab9
lab2 lab4 lab6 lab8 README.md
hby@ubuntu2004:~/lab1_ws/VNAV-labs$ cd lab1 # 进入第一个实验
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ ls
main.cpp random_vector.cpp random_vector.h
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ cat main.cpp
#include <cstdlib> // required for srand
#include <iostream>
#include "random_vector.h"

int main(){
    // initialize rand with constant seed for reproducibility
    std::srand(314159);

    RandomVector rv(20);
    rv.print();
    std::cout << "Mean: " << rv.mean() << std::endl;
    std::cout << "Min: " << rv.min() << std::endl;
    std::cout << "Max: " << rv.max() << std::endl;

    std::cout << "Histogram:" << std::endl;
    rv.printHistogram(5);
    std::cout << std::endl;
}
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ g++ main.cpp random_vector.cpp -o lab1_test # 编译两个cpp文件, 生成可执行程序
random_vector.cpp: In member function 'double RandomVector::mean()':
random_vector.cpp:14:1: warning: no return statement in function returning non-void [-Wreturn-type]
  14 | }
    | ^
random_vector.cpp: In member function 'double RandomVector::max()':
random_vector.cpp:18:1: warning: no return statement in function returning non-void [-Wreturn-type]
  18 | }
    | ^
random_vector.cpp: In member function 'double RandomVector::min()':
random_vector.cpp:22:1: warning: no return statement in function returning non-void [-Wreturn-type]
  22 | }
    | ^
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ ./lab1_test # 运行 Lab1 程序
Mean: 1
Min: 3.03428e-86
Max: 3.03428e-86
Histogram:
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$
```

Figure 14: Reading the lab1 sources

The personal submission repository `hbyhby123/vnav-personal` was created on GitHub and cloned to `~/vnav-personal`, with a `lab1` folder holding all the exercise solutions. Because this is an on-campus course, a public `github.com` repository replaces the `github.mit.edu` organization repository. The main branch contains the following pushed revisions:

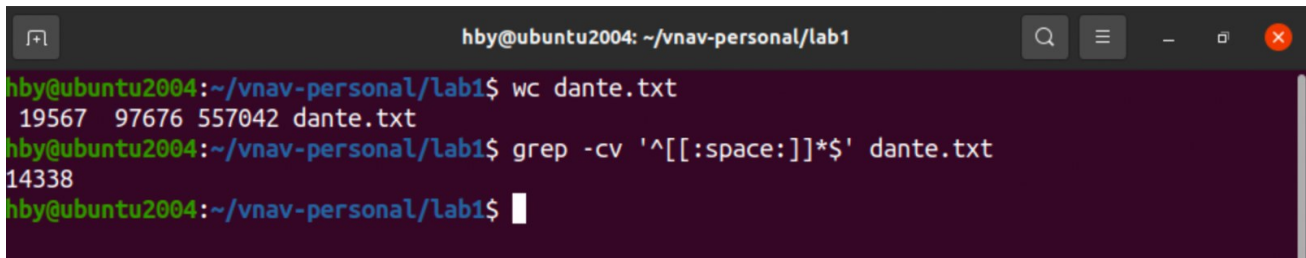
Commit	Content
<code>bf9c003</code>	Completed all Lab 1 exercise files
<code>da39796</code>	Added the <code>RandomVector</code> implementation
<code>ad2d95a</code>	Removed the <code><algorithm></code> header, hand-written max/min loops
<code>3f7f490</code>	Added <code>exercise1.txt</code> , fixed histogram bins and output direction
<code>55e790b</code>	Added the four <code>lab1_demo</code> project files
<code>c6d80db</code>	Appended the five fortune outputs

A final `git status` check shows only build artifacts (`RandomVector/random_vector` and `lab1_demo/build/`) untracked; they were deliberately not committed.

Part Four: Shell Exercise 1 — `dante.txt` Statistics

The task is to download the course-specified `dante.txt` with `wget`, create `exercise1.txt` in `~/vnav-personal/lab1` answering three questions — how many lines, how many words, how many non-blank lines — and push the file to git.

```
wget https://raw.githubusercontent.com/dlang/dmd/master/druntime/benchmark/extra-files/dante.txt
wc dante.txt
grep -cv '^[[:space:]]*$' dante.txt
```



A terminal window titled 'hby@ubuntu2004: ~/vnav-personal/lab1' shows the following commands and output:

```
hby@ubuntu2004:~/vnav-personal/lab1$ wc dante.txt
19567  97676 557042 dante.txt
hby@ubuntu2004:~/vnav-personal/lab1$ grep -cv '^[[:space:]]*$' dante.txt
14338
hby@ubuntu2004:~/vnav-personal/lab1$
```

Figure 15: `dante.txt` statistics

Figure 09: `wc` reports `19567 97676 557042 dante.txt` (lines, words, bytes) and `grep -cv` reports 14338 non-blank lines; the same value was double-checked with `grep | wc -l`.

`exercise1.txt`:

```
Lines: 19567
Words: 97676
Non-blank lines: 14338
```

The result was committed and pushed (commit `3f7f490`). `wc -l` counts lines and `wc -w` counts words; for “non-blank lines” the blank-only lines must first be filtered out with `grep` and then counted with `wc -l`.

Part Four: Shell Exercise 2 — Appending fortune Outputs

The task is to install `fortune-mod` with `apt`, run `fortune` once to see a quote, then run `fortune` five more times, each time **appending** the output to `~/vnav-personal/lab1/fortunes.txt` (the file must not be recreated), and push the file to git.

```
sudo apt install fortune-mod
```

```
fortune
```

```
fortune >> ~/vnav-personal/lab1/fortunes.txt # append #1
```

```
fortune >> ~/vnav-personal/lab1/fortunes.txt # append #2
```

```
fortune >> ~/vnav-personal/lab1/fortunes.txt # append #3
```

```
fortune >> ~/vnav-personal/lab1/fortunes.txt # append #4
```

```
fortune >> ~/vnav-personal/lab1/fortunes.txt # append #5
```

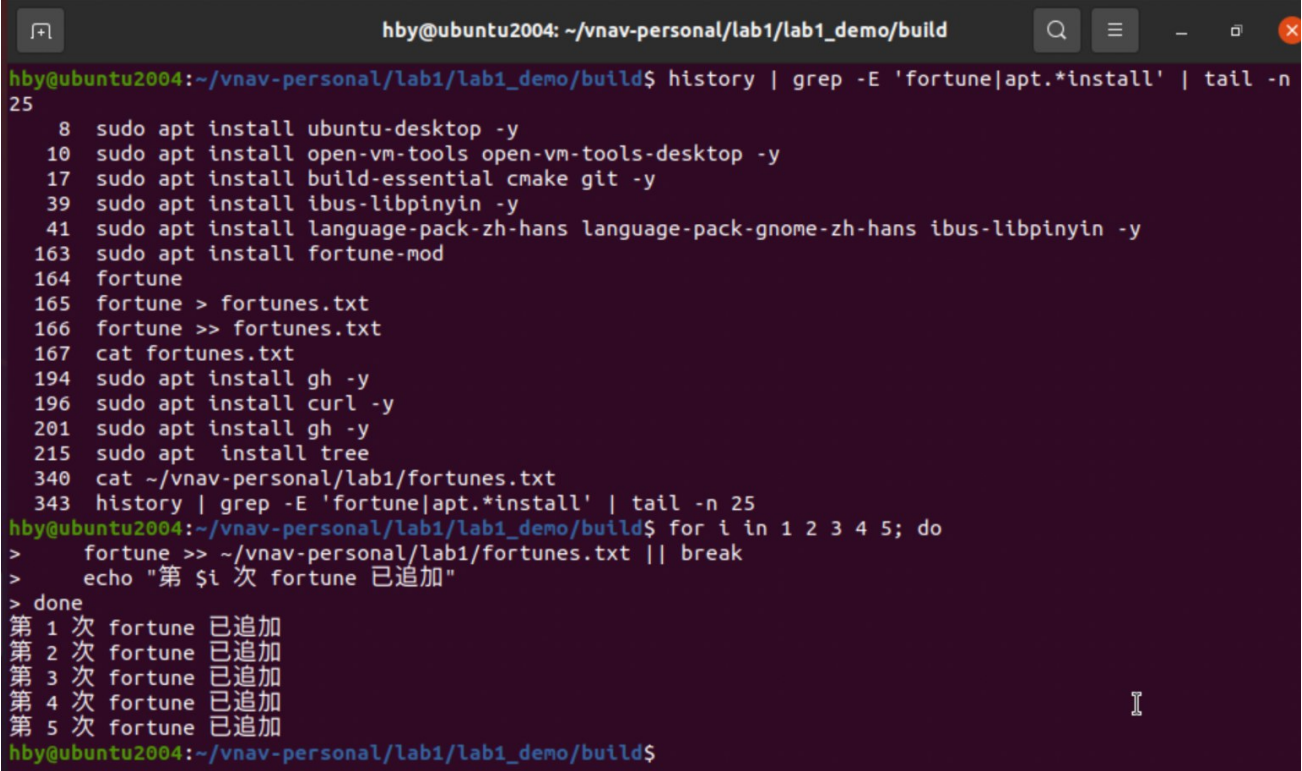
A terminal window titled 'hby@ubuntu2004: ~/vnav-personal/lab1/lab1_demo/build'. The user runs 'history | grep -E 'fortune|apt.*install' | tail -n 25'. The output shows a list of commands including 'sudo apt install ubuntu-desktop -y', 'sudo apt install open-vm-tools open-vm-tools-desktop -y', 'sudo apt install build-essential cmake git -y', 'sudo apt install ibus-libpinyin -y', 'sudo apt install language-pack-zh-hans language-pack-gnome-zh-hans ibus-libpinyin -y', 'sudo apt install fortune-mod', 'fortune', 'fortune > fortunes.txt', 'fortune >> fortunes.txt', 'cat fortunes.txt', 'sudo apt install gh -y', 'sudo apt install curl -y', 'sudo apt install gh -y', 'sudo apt install tree', 'cat ~/vnav-personal/lab1/fortunes.txt', and 'history | grep -E 'fortune|apt.*install' | tail -n 25'. The user then runs a loop: 'for i in 1 2 3 4 5; do fortune >> ~/vnav-personal/lab1/fortunes.txt || break; echo "第 \$i 次 fortune 已追加"; done'. The output of the loop shows five successful appends: '第 1 次 fortune 已追加', '第 2 次 fortune 已追加', '第 3 次 fortune 已追加', '第 4 次 fortune 已追加', and '第 5 次 fortune 已追加'. The prompt returns to 'hby@ubuntu2004:~/vnav-personal/lab1/lab1_demo/build\$'.

Figure 16: fortune installation and calls in shell history

Figure 10: Shell history shows `sudo apt install fortune-mod` and the `fortune` calls; all five appends succeeded and the previous content was preserved.

Excerpt of `fortunes.txt` (old content kept, new quotes appended at the end):

You will have a long and unpleasant discussion with your supervisor.

A Tale of Two Cities LITE(tm)

-- by Charles Dickens

...

Beware of a tall black man with one blond shoe.

April 1

-- Mark Twain, "Pudd'nhead Wilson's Calendar"

...

The file was committed and pushed (commit c6d80db). The key lesson: `>` **overwrites** the file while `>>` **appends**; since every new quote must keep the previous ones, `>>` is required.

Part Four: C++ Warm-Up Exercises

The answers below match `cpp-warmup.txt`, with the numeric-type differences stated more precisely than in the original file.

Operators

1. After the following code, what are the values of `i` and `j`?

```
int i = 0, j;
j = ++i;
j = i++;
```

Answer: initially `i = 0`. `j = ++i` increments first (`i = 1`) and assigns `j = 1`; `j = i++` assigns first (`j = 1`) and increments after (`i = 2`). Final values: **`i = 2, j = 1`**.

2. What does the following code print?

```
int i = 42;
std::string output = (i < 42) ? "a" : "b";
std::cout << output << std::endl;
```

Answer: `i < 42` is false, so the conditional expression takes the second branch and prints **b**.

References and Pointers

1.

```
int i;
int& ri = i;
i = 5;
ri = 10;
std::cout << i << " " << ri << std::endl;
```

Answer: `ri` is a reference to `i`, i.e. the same object. After `ri = 10`, `i` is also 10, so the program prints **10 10**.

2.

```
int i = 42;
int* j = &i;
*j = *j**j;
std::cout << *j << std::endl;
```

Answer: `*j**j` parses left-to-right as $(*j) * (*j) = 42 \times 42 = 1764$. `*j` and `i` are the same object, so `i` also becomes 1764.

3.

```
int i[4] = {42,24,42,24};
*(i+2) = *(i+1)-i[3];
std::cout << *(i+2) << std::endl;
```

Answer: `*(i+2)` is `i[2]`, `*(i+1)` is `i[1] = 24` and `i[3] = 24`, so `i[2] = 24 - 24 = 0` and the program prints **0**. An array name decays to a pointer to its first element, so `i[k]` is the same as `*(i+k)`.

4.

```
void reset(int &i) {
    i = 0;
}
int j = 42;
reset(j);
std::cout << j << std::endl;
```

Answer: the parameter of `reset` is a reference, so `i = 0` inside the function modifies the caller's `j` directly and the program prints **0**. Reference parameters avoid copies and let a function modify the argument.

Numbers

1. Differences between `int`, `long`, `long long` and `short`?

Answer: they are all integer types and differ in storage size and representable range. The standard only guarantees `sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)` with minimum widths of 16, 16, 32 and 64 bits respectively, so `long` must not be assumed to be a fixed 4 bytes. On common ARM64 Linux they are 2, 4, 8 and 8 bytes. Pick the smallest type whose range covers the values you need.

2. Differences between `float` and `double`? Value of `i` after the snippet?

```
int i;
i = 3.14;
```

Answer: `float` is single precision (usually 32 bits, about 6–7 decimal significant digits), `double` is double precision (usually 64 bits, about 15–16 digits), with higher precision and range. Assigning 3.14 to an `int` truncates toward zero, so `i = 3`.

3. Differences between `unsigned` and `signed`? Value of `c` (8-bit `char`)?

```
unsigned char c = -1;
```

Answer: signed types represent negative, zero and positive values; unsigned types represent only non-negative values and wrap around modulo 2^n . An 8-bit `unsigned char` ranges from 0 to 255; converting -1 to it is -1 mod 256, so `c = 255`.

4. Value of `i` after the snippet?

```
int i = 42;
if (i) {
    i = 0;
} else {
    i = 43;
}
```

Answer: `i = 42` is non-zero, hence true, so the if-branch runs and `i = 0`.

All answers were cross-checked against `cpp-warmup.txt` and agree.

Part Four: RandomVector Implementation

The project lives in `lab1/RandomVector`. The header `random_vector.h` declares the interface and `random_vector.cpp` contained TODOs that had to be completed. `main.cpp` fixes the seed with `std::srand(314159)`, so every run produces the same 20 values, which makes the code easy to check.

```
// random_vector.h (interface)
#include <iostream>
#include <vector>

class RandomVector{
    std::vector<double> vect;

public:
    RandomVector(int size, double max_val = 1);
    void print();
    double mean();
    double max();
    double min();
    void printHistogram(int bins);
};
```

Function	Purpose
<code>RandomVector(int size, double max_val=1)</code>	Generate <code>size</code> random values in <code>[0, max_val]</code> and store them in <code>vect</code>
<code>print()</code>	Print all values
<code>mean()</code>	Return the mean of the values
<code>max() / min()</code>	Return the maximum / minimum value
<code>printHistogram(int bins)</code>	Count values into <code>bins</code> equal-width bins between <code>min()</code> and <code>max()</code> and print the vertical histogram

Core implementation:

```
RandomVector::RandomVector(int size, double max_val)
{
    for(int i = 0; i < size; i++)
    {
        double value = ((double)rand() / RAND_MAX) * max_val;
        vect.push_back(value);
    }
}

double RandomVector::mean()
{
    double sum = 0;
    for(double v : vect) sum += v;
    return sum / vect.size();
}

double RandomVector::max()
{
    double max_value = vect[0];
    for(double v : vect)
        if(v > max_value) max_value = v;
    return max_value;
}

double RandomVector::min()
{
    double min_value = vect[0];
    for(double v : vect)
        if(v < min_value) min_value = v;
    return min_value;
}
```

Histogram core logic:

```
void RandomVector::printHistogram(int bins)
{
    if (bins <= 0 || vect.empty()) return;

    double low = min();
    double high = max();
    std::vector<int> histogram(bins, 0);

    for (double v : vect)
    {
        int index = 0;
```

```

    if (high > low)
    {
        index = static_cast<int>((v - low) / (high - low) * bins);
        if (index >= bins) index = bins - 1;
    }
    histogram[index]++;
}

int height = 0;
for (int count : histogram)
    if (count > height) height = count;

// print row by row from the top: a bin prints *** while its count reaches the row
for (int row = height; row > 0; row--)
{
    for (int i = 0; i < bins; i++)
    {
        if (histogram[i] >= row) std::cout << "***";
        else std::cout << "   ";
        if (i < bins - 1) std::cout << " ";
    }
    std::cout << std::endl;
}
}

```

Corrections made during development:

1. **Removed the `<algorithm>` header and its function calls** (the course requires not using them); max/min are computed with plain loops (commit ad2d95a).
2. **Bins changed from a fixed 0–1 range to `min()–max()`** so the histogram uses equal-width bins between the actual minimum and maximum, as the spec requires (commit 3f7f490).
3. **Output direction changed from horizontal to vertical** to match the teacher's example: one row per level, *** per filled cell.
4. **Boundary handling:** return immediately when `bins <= 0` or the vector is empty; when all values are equal (`high == low`) everything falls into the first bin; the maximum value falls into the last bin so the index never equals bins.
5. mean/max/min assume a non-empty vector; this run contains 20 elements.

Part Four: Build, Run and Results

The program was first compiled directly with g++ and then built with CMake in a `build` directory and run as `lab1_test`.

```

g++ -std=c++11 -Wall -pedantic -o random_vector main.cpp random_vector.cpp
./random_vector

cmake_minimum_required(VERSION 3.10)
project(vnav_lab1)
add_executable(lab1_test main.cpp random_vector.cpp)

mkdir -p build && cd build
cmake .. && make && ./lab1_test

```

Figure 11: Writing CMakeLists.txt and building/running the lab in the build directory.

Final output (fixed seed `std::srand(314159)`, identical on every run):

```

0.458724 0.779985 0.212415 0.0667949 0.622538 0.999018 0.489585 0.460587 0.0795612
0.185496 0.629162 0.328032 0.242169 0.139671 0.453804 0.083038 0.619352 0.454482

```

```
hby@ubuntu2004: ~/lab1_ws/VNAV-labs/lab1/build

hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ cat > CMakeLists.txt <<'EOF'
> cmake_minimum_required(VERSION 3.10)
>
> project(vnav_lab1)
>
> add_executable(lab1_test main.cpp random_vector.cpp)
>
> EOF
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ # 创建CMake配置文件
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ mkdir build
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ # 创建编译目录
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1$ cd build
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$ # 进入编译目录
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$ cmake ..
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/hby/lab1_ws/VNAV-labs/lab1/build
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$ # 根据CMakeLists 生成编译文件
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$ make
Scanning dependencies of target lab1_test
[ 33%] Building CXX object CMakeFiles/lab1_test.dir/main.cpp.o
[ 66%] Building CXX object CMakeFiles/lab1_test.dir/random_vector.cpp.o
[100%] Linking CXX executable lab1_test
[100%] Built target lab1_test
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$ # 编译程序
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$
hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$ ./lab1_test
0.193146 0.935807 0.8478698 0.831597 0.539111 0.8278694 0.93919 0.544454 0.893339 0.693182 0.415163 0.454242 0.320536 0.0766892 0.809584 0.93073 0.982655 0.102505 0.716924 0.706215
Mean: 0.55804
Min: 0.0278694
Max: 0.982655
Histogram:
0: *****
1: *
2: ****
3: ***
4: *****

hby@ubuntu2004:~/lab1_ws/VNAV-labs/lab1/build$
```

Figure 17: CMake build in the build directory

```

0.477426 0.0904966
Mean: 0.393617
Min: 0.0667949
Max: 0.999018
Histogram:
***      ***
***      ***
***      ***
***      ***
***      ***
***      ***
***      *** ***
*** ***  *** *** ***

```

Metric	Result
Number of values	20
Mean	0.393617
Min	0.0667949
Max	0.999018
Number of bins	5
Bin counts (bins 0–4)	8, 1, 8, 2, 1
Histogram height	8

Reproducibility: because the seed is fixed to `std::srand(314159)`, every run prints the same 20 values, which makes the output easy to check. The output matches the course website example item by item (Mean 0.393617, Min 0.0667949, Max 0.999018), confirming that the implementation behaves exactly as expected.

Results Analysis and Problems Solved

Histogram result

The range from the actual minimum 0.0667949 to the actual maximum 0.999018 is split into five equal-width bins:

Bin	Range	Count
0	[0.067, 0.253)	8
1	[0.253, 0.440)	1
2	[0.440, 0.626)	8
3	[0.626, 0.813)	2
4	[0.813, 0.999]	1

Figure 12: Bar chart of the final output. The counts sum to $8+1+8+2+1 = 20$, equal to the vector length, which verifies the binning logic; with only 20 samples the histogram is not expected to be smooth.

An earlier run with an unfixed seed

One actual run recorded during the experiment used a `main.cpp` without the fixed seed and printed Mean 0.55804, Min 0.0278694, Max 0.982655 with bin counts 5, 1, 4, 3, 7 (still summing to 20), compiled without warnings or errors.

Figure 13: An earlier actual run (unfixed seed) whose output differs from the submitted version.

Different random values naturally give a different mean and histogram; that is expected. However, without a fixed seed every run differs, which makes self-checking and grading hard. The submitted version therefore restores the

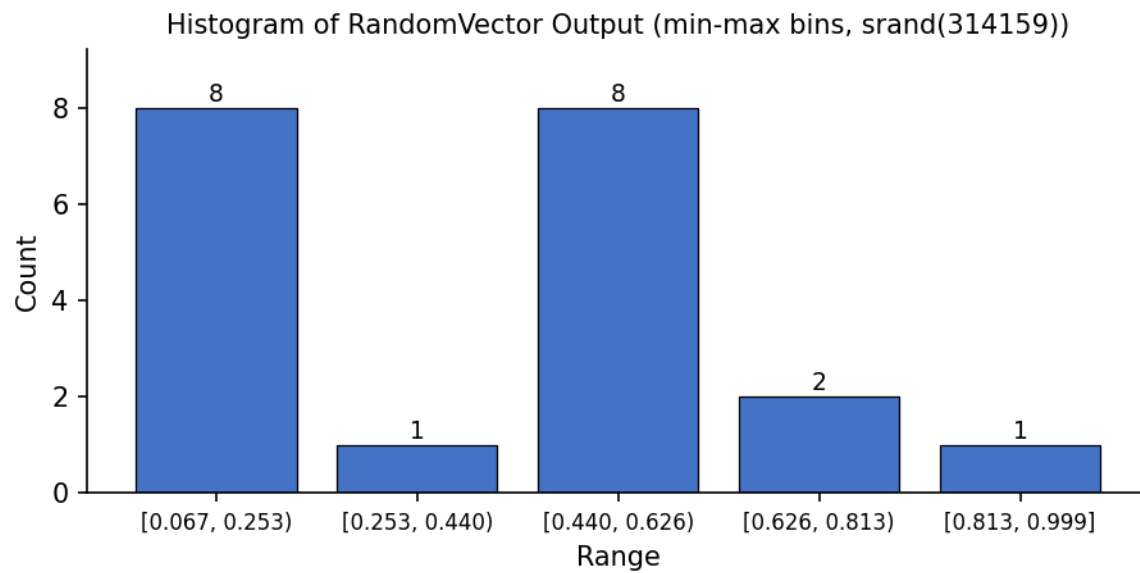


Figure 18: Histogram of the final output

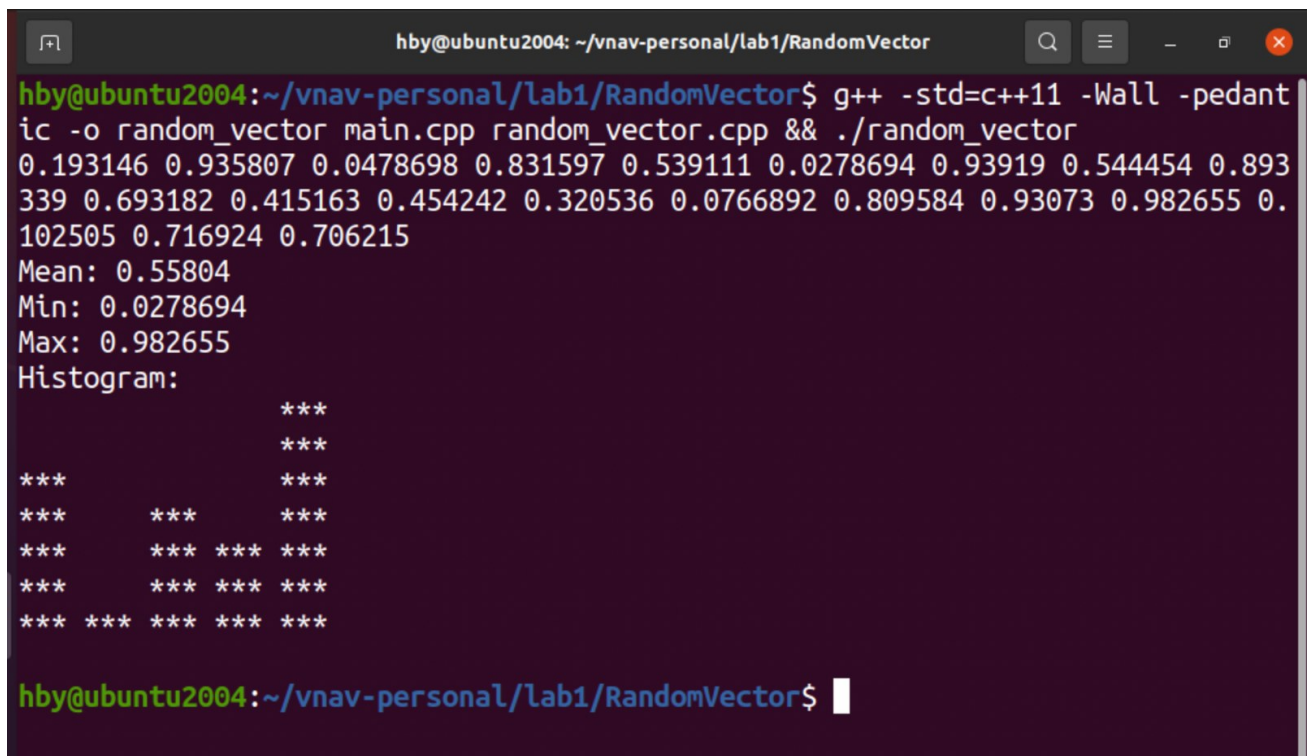


Figure 19: An earlier run without the fixed seed

original `std::srand(314159)` from `main.cpp`: the output is stable at Mean 0.393617 (see the previous section) and matches the course example. The exact histogram shape is not required to equal the teacher's example; only the counting logic matters.

Problems encountered and solutions

Problem	Cause / symptom	Solution
<code>file:/cdrom</code> error from <code>apt update</code>	A leftover CD-ROM entry in the sources	Removed the <code>cdrom</code> entries and re-ran <code>sudo apt update</code> ; later backed up and switched to the Tsinghua mirror
Input-method shortcut clash between Mac and Ubuntu	Ctrl+Space taken by macOS	Changed the shortcut; used an English environment during the lab
<code>git clone</code> asked for a GitHub username/password	A malformed repository address was used first	Used the standard HTTPS address
Compiling before completing the TODOs	<code>no return statement</code> warnings and abnormal output	Implemented the functions per the interface and rebuilt
Only a text console after the first boot	The system first boots to the terminal login prompt	Logged in, finished the configuration and rebooted into the desktop
Histogram bins and direction did not match the spec	Early version used a fixed 0–1 range and horizontal output	Switched to <code>min()–max()</code> equal-width bins and vertical <code>***</code> output (commit <code>3f7f490</code>)
Used the <code><algorithm></code> header	The course requires not using its functions	Removed it and hand-wrote the <code>max/min</code> loops (commit <code>ad2d95a</code>)
<code>exercise1.txt</code> missing	Not created yet	Added the <code>wc</code> statistics and committed (commit <code>3f7f490</code>)
Random output not reproducible	The seed was not fixed in <code>main.cpp</code>	Restored the course-original <code>srand(314159)</code> ; the output is stable and matches the course example

Conclusion

In this lab I completed the whole pipeline: installing Ubuntu 20.04.5 ARM64 in VMware Fusion and switching to the Tsinghua mirror; installing and verifying build-essential, CMake and Git; practicing basic Linux commands (`ls`, `cd`, `mkdir`, `echo`, `cat`, `wc`, `grep`, ...); Git version management (`init`, `add`, `commit`, `log`, `clone`, `push`) with the course repository and a personal submission repository; single-file `g++` compilation and a multi-file CMake project (`lab1_demo` with a library linked to an executable); the Shell exercises (`dante.txt`: 19567 lines, 97676 words, 14338 non-blank lines; five appended fortune outputs); the C++ warm-up questions committed as `cpp-warmup.txt`; and the complete `RandomVector` class with six member functions — no `<algorithm>` dependency, `min()–max()` equal-width bins, vertical histogram output. The final program deterministically prints 20 random values, the mean, the `min/max` and the histogram and matches the course example, so all Lab 1 goals are achieved.

The revision also fixed the missing `exercise1.txt`, the histogram range/direction issues and the missing multi-file build evidence. Two practical lessons: `>` overwrites while `>>` appends, and CMake's `add_library/target_link_libraries` express the linking relationship explicitly. Before running Git commands, double-check the current directory, and check boundary conditions against the interface specification rather than only testing whether the program runs.